Experiment No. 8
Implementation of a Regression based recommendation system
Date of Performance:
Date of Submission:
Marks:
Sign:

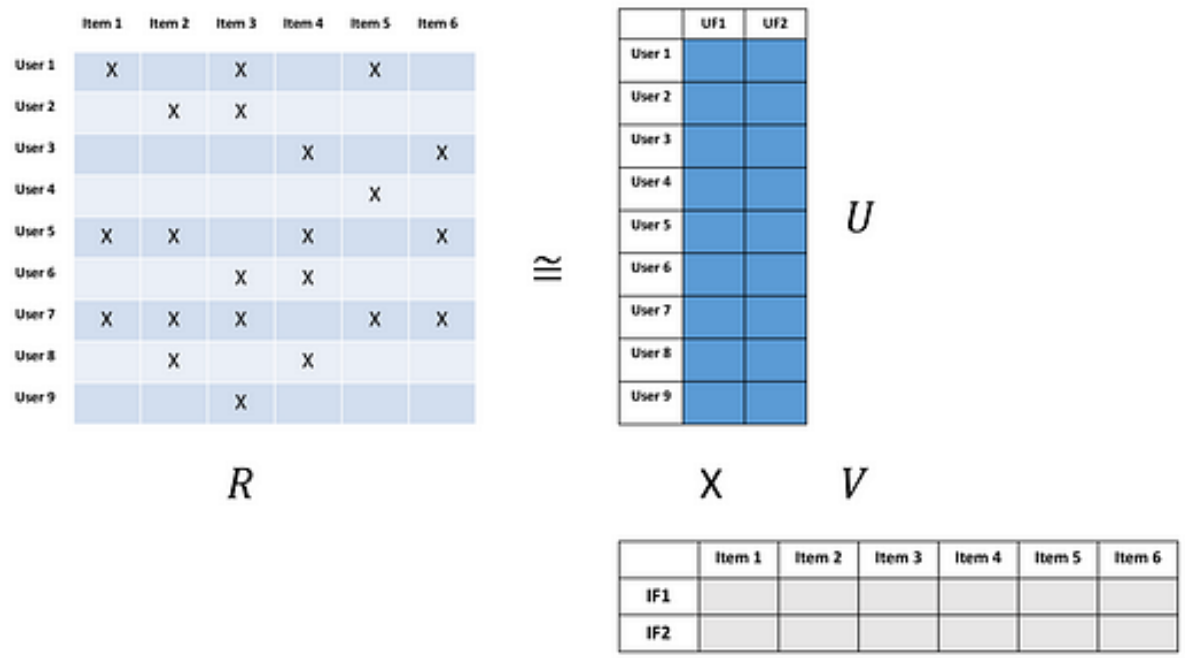
Aim: Implementation of a Regression based recommendation system.

Objective: Able to design a regression based recommendation system with respective implementation.

Theory: Often, matrix factorization is applied in the realm of dimensionality reduction, where we are trying to reduce the number of features while still keeping the relevant information. This is the case with principal component analysis (PCA) and the very similar singular value decomposition (SVD).

Essentially, can we take a large matrix of user/item interactions and figure out the latent (or hidden) features that relate them to each other in a much smaller matrix of user features and item features? That's exactly what ALS is trying to do through matrix factorization.

As the image below demonstrates, let's assume we have an original ratings matrix RR of size MxNMxN, where MM is the number of users and NN is the number of items. This matrix is quite sparse, since most users only interact with a few items each. We can factorize this matrix into two separate smaller matrices: one with dimensions MxKMxK which will be our latent user feature vectors for each user (U)(U) and a second with dimensions KxNKxN, which will have our latent item feature vectors for each item (V)(V). Multiplying these two feature matrices together approximates the original matrix, but now we have two matrices that are dense including a number of latent features KK for each of our items and users.



In order to solve for UU and VV, we could either utilize SVD (which would require inverting a potentially very large matrix and be computationally expensive) to solve the factorization more precisely or apply ALS to approximate it. In the case of ALS, we only need to solve one feature vector at a time, which means it can be run in parallel! (This large advantage is probably why it is the method of choice for Spark). To do this, we can randomly initialize UU and solve for VV. Then we can go back and solve for UU using our solution for VV.

Implementation:

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression

# Sample User-Item Ratings Data
data = {
   'user_id': [1, 1, 2, 2, 3, 3, 4, 4],
   'item_id': [1, 2, 1, 3, 2, 4, 3, 4],
   'rating':  [5, 3, 4, 2, 4, 5, 3, 4]
}

df = pd.DataFrame(data)

print("Dataset:")
print(df)

# -------------------------------
# Step 1: Create Features
# -------------------------------
# Using user_id and item_id as features
X = df[['user_id', 'item_id']]
y = df['rating']
```

```
# -------------------------------
# Step 2: Train Regression Model
# -------------------------------
model = LinearRegression()
model.fit(X, y)

# -------------------------------
# Step 3: Predict Ratings
# -------------------------------
# Predict for all user-item pairs
users = df['user_id'].unique()
items = df['item_id'].unique()

predictions = []

for user in users:
    for item in items:
        pred_rating = model.predict([[user, item]])[0]
        predictions.append([user, item, pred_rating])

pred_df = pd.DataFrame(predictions, columns=['user_id', 'item_id',
'predicted_rating'])

print("\nPredicted Ratings:")
print(pred_df)

# -------------------------------
# Step 4: Recommend Items
# -------------------------------
# Recommend top items for a specific user (e.g., User 2)
user_id = 2
```

```
# Remove already rated items
rated_items = df[df['user_id'] == user_id]['item_id'].tolist()

recommendations = pred_df[(pred_df['user_id'] == user_id) &
                          (~pred_df['item_id'].isin(rated_items))]

# Sort by predicted rating
recommendations = recommendations.sort_values(by='predicted_rating',
ascending=False)

print("\nRecommended Items for User", user_id)
print(recommendations)
```

Conclusion:

In this experiment, a Regression-Based Recommendation System was implemented using a Linear Regression model. The system predicted user-item ratings based on features such as user ID and item ID. By learning patterns from existing ratings, the model was able to estimate unknown ratings and recommend items accordingly. Although simple, regression-based methods provide a foundational understanding of predictive recommendation systems. However, they may not capture complex user-item relationships as effectively as advanced techniques like matrix factorization or deep learning models.